

DESIGN REPORT

This project demonstrates a working concurrent task dispatcher using Rust's own standard concurrency tools. System simulates how tasks arrive over time, will enter a queue, and are scheduled by a dispatcher which are now executed by a bounded worker pool. This project's main purpose is to focus on demonstrating the understanding of the concepts learned in class about concurrency, task scheduling, queue-based problems, worker pools, synchronization, and metrics.

This program acts as a simulation to mimic close to what might be seen in a real operating system. It mimics realistic scheduling behavior where a task generator creates incoming work and sends those tasks into the system. This also shows it in a way that it contains 8 'workers' and a total of 1000 tasks that will be generated automatically using the random fixed seed so that these experiments (Balanced or not) are given.

What is NEEDED in the system was a

- Task generator thread
- Dispatcher thread
- 8 workers thread (as required in the exalidraw document)
- Shared metrics collection
- And Queue-Based communication channels

```
37
38  #[derive(Clone, Debug)]
39  struct Task {
40      id: usize,
41      kind: TaskKind,
42      arrival_time: Instant,
43      duration_ms: u64,
44      cpu_cost: u32,
45  }
46
```

The given task includes two types which will behave differently.

CPU focused tasks are good for simulating heavier CPU usage while IO tasks simulate lighter CPU usage. These tasks will be used and given the concurrency tools from the Rust library will work between threads.

Mpsc channels

A main important tool is “mpsc” which can be found in the top of the code to implement it into the program.

use std::sync::{mpsc, Arc, Mutex}; // mpsc is for communication over channels...

Channels are much easier to use when it comes to transferring ownership of tasks safely between threads. It is preferable as well, and including threads differently is time-consuming and may lead to a possible problem in the future. Which is why mpsc is used instead. It allows the generator to send tasks to the dispatcher, and the dispatcher sends tasks to workers in the simulation, allowing it to avoid any unsafe access to task data.

Arc<Mutex<Metrics>>

These are shared between worker threads. Since this program will simulate many workers doing tasks and updating the progress at the same time, there should be something that prevents any problems. This is where the metrics structure is protected using a Metrex. Arc allows many multiple threads to safely share the ownership of the metrics.

There are other things necessary to make the programming run smoothly, such as...

- AtomicBool (for shutting down/signaling)
 - This focuses on shutdown signaling where worker threads and the dispatcher repeatedly checks whether the program should stop running. This is necessary to avoid any infinite loops or lack of proper termination when the workers are done with the tasks.
- Worker Threads
 - For the system to progress, it uses a bounded worker pool (with the const WORKER_COUNT that contains 8) The purpose is to wait for tasks and execute them, along with updating the progress when finished and continuing for more work if given.
- Recv_timeout()
 - Allows threads to check any shutdown conditions, preventing any hanging issues.

With this, it is possible for the generator to create task over time (task manager) which are sent into a central queue. The fn dispatcher THEN receives tasks and picks (which worker handles what), where the workers now proceed and program simulates concurrency.

After all the tasks are completed, the system will shut down smoothly without issues.

FIFO and Optimized scheduling were implemented for the project.

FIFO: is where the policy distributes tasks evenly across workers in order (turnaround).

Optimized policy attempts to separate CPU heavy tasks from IO ones. This is shown through (Fifo 70% IO / 30% CPU) and Optimized, //Optimized 80%.

Lastly, to properly demonstrate the difference between FIFO and optimized, it is important to run these where FIFO uses.

```
4 match policy {
3   Policy::FIFO => {
4     worker_senders[rr % WORKER_COUNT].send(task).unwrap();
5     rr += 1;
6   }
7
8   Policy::Optimized => {
9     // CPU tasks will now spread evenly go IO
10    let target = match task.kind {
11      TaskKind::CPU => 0,
12      TaskKind::IO => (rr + 1) % WORKER_COUNT,
13    };
14
15    worker_senders[target].send(task).unwrap();
16    rr += 1;
17  }
18 }
```

•

FIFO'S RESULTS | OPTIMIZED RESULTS

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORT
Worker 0 finished task 976
Worker 0 finished task 984
Worker 1 finished task 985
Worker 5 finished task 997
Worker 0 finished task 992
Worker 1 finished task 993

RESULTS ARE:
Tasks completed: 1000
CPU tasks: 311
IO tasks: 689
Avg wait: 6475 ms
Avg turnaround: 6661 ms
Total runtime (makespan): 23738 ms
=====
```

As shown here, the main difference is the avg wait, turnaround, and the required total_runtime of each policy.

```
Worker 0 finished task 995  
Worker 0 finished task 997  
Worker 0 finished task 998  
Worker 0 finished task 999
```

```
RESULTS ARE:  
Tasks completed: 1000  
CPU tasks: 311  
IO tasks: 689  
Avg wait: 15653 ms  
Avg turnaround: 15839 ms  
Total runtime (makespan): 83038 ms  
=====
```